

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 05 of 11

Shared Service Components

Product	BrokerVerse
Document	05 Shared Service Components
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 05 Shared Service Components
Document ID	BV-SA-05
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
2 Backend shared services.....	6
2.1 Overview.....	6
2.2 Configuration (src/config.js).....	6
2.3 Route registry (lib/registry.js).....	7
2.4 Authentication and authorisation (lib/auth.js and related).....	7
2.5 Record scope (lib/scope.js).....	7
2.6 Settings (lib/settings.js).....	8
2.7 Numbering.....	8
2.8 Audit (lib/audit.js).....	8
2.9 Notifications and audience.....	8
2.10 E-mail outbox (lib/mailer.js).....	8
2.11 Documents (PDF, XLSX, CSV).....	9
2.12 Reports engine (modules/reports).....	9
2.13 File storage and signed links.....	9
2.14 Scheduler (jobs/scheduler.js).....	10
2.15 Rate limiting and upload limits.....	10
2.16 Dates, money and validation.....	10
3 Front-end shared components.....	12
4 Related documents.....	13
5 Glossary.....	14

1 Introduction

1.1 Purpose and scope

This document describes the shared service components of BrokerVerse: the backend services in `backend/src/lib`, `backend/src/jobs` and `backend/src/tools` that every business module uses, and the shared front-end components and utilities in `brokerverse/src`. For each component it gives the responsibility, the main interface, the configuration it reads and the points to observe when extending or operating it. New modules should use these components rather than re-implementing them.

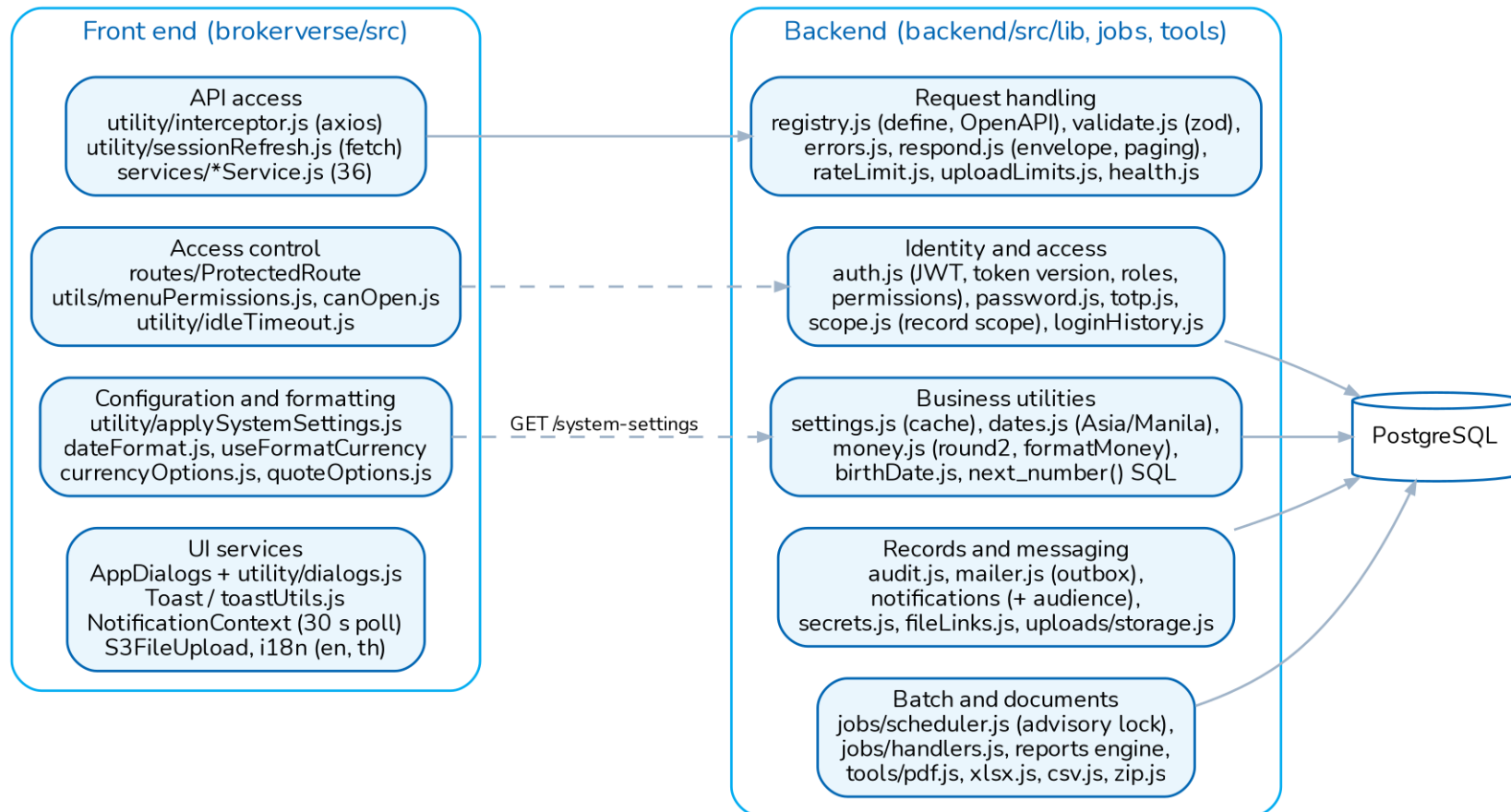


Figure 1. Shared service components of the front end and the backend

2 Backend shared services

2.1 Overview

Component	Location	Responsibility
Configuration	src/config.js	Environment configuration and the production start-up check
Route registry	lib/registry.js	Declares every endpoint, enforces sign-in, records roles / permissions for documentation
Authentication and authorisation	lib/auth.js, lib/password.js, lib/totp.js, lib/loginHistory.js	Tokens, session revocation, roles and permissions, password policy, two-factor, sign-in log
Record scope	lib/scope.js	Agents see only their own book
Settings	lib/settings.js	Configuration-driven behaviour from app_settings with an in-process cache
Numbering	SQL next_number() and helpers	Yearly document numbers with configurable prefixes
Audit	lib/audit.js	Audit log of every change
Notifications	modules/notifications/ router.js notify()	In-app notifications to a user or to a permission audience
E-mail outbox	lib/mailer.js	Queue and deliver e-mail
Documents	modules/documents, tools/pdf.js, tools/xlsx.js, tools/csv.js, tools/zip.js	PDF, XLSX and CSV generation
Reports engine	modules/reports/engine.js, queries.js, service.js	Parameterised report SQL, on-screen runs, files, schedules
File storage and signed links	modules/uploads/ storage.js, fileType.js, lib/fileLinks.js, lib/secrets.js	S3-style key space on disk, type checking, signed expiring URLs
Scheduler	jobs/scheduler.js, jobs/handlers.js	Cron jobs with PostgreSQL advisory locks and run history
Rate limiting and upload limits	lib/rateLimit.js, lib/uploadLimits.js	Sign-in and global API limits; multipart and import limits
Dates and money	lib/dates.js, lib/money.js, lib/birthDate.js	Business dates in the configured time zone; rounding and formatting
Validation, errors and responses	lib/validate.js, lib/errors.js, lib/respond.js	zod validation, error envelope, success envelope and paging
Health	lib/health.js, modules/system	Readiness, liveness, version
Database access	db/pool.js	Pool, query, one, many, withTransaction

2.2 Configuration (src/config.js)

buildConfig() reads the environment once (with .env through dotenv in development): PORT (8000), DATABASE_URL, JWT_SECRET, JWT_ACCESS_TTL_SECONDS (1,800), JWT_REFRESH_TTL_SECONDS (2,592,000), DATA_ENCRYPTION_KEY, CORS_ORIGINS, UPLOAD_DIR, PUBLIC_BASE_URL, SMTP_URL, LOG_LEVEL, FILE_URL_TTL_SECONDS (1,800), JSON_BODY_LIMIT (2 MB), UPLOAD_MAX_MB (10), UPLOAD_MAX_FILES (10), IMPORT_MAX_MB (10), IMPORT_MAX_INFLATED_MB (50), IMPORT_MAX_ROWS (20,000). Other variables read

directly: `SEED_SAMPLE_DATA`, `SCHEDULER_ENABLED`, `ADMIN_PASSWORD`, `HEALTH_DB_TIMEOUT_MS` (2,000), `SHUTDOWN_TIMEOUT_MS` (25,000), `GIT_COMMIT`. With `NODE_ENV=production`, `assertProductionConfig()` stops the process when a secret is missing, a placeholder, shorter than 32 characters or reused, when `CORS_ORIGINS` is `*`, or when `PUBLIC_BASE_URL` is missing or local.

2.3 Route registry (lib/registry.js)

Every module creates its router with `moduleRouter(module, prefix)` and declares each endpoint with `define({ method, path, summary, auth, roles, permissions, screen, request, query, response, middleware, handler }).define()`:

- places `requireAuth` first unless the route is declared `auth: false`, so no route can be left unauthenticated by omission;
- wraps the async handler so errors reach the error middleware;
- records the route, the roles and permissions required by its middleware, the front-end screen and example payloads in `ROUTES`, from which `npm run export:api` generates the OpenAPI document, the Postman collection and the Excel / CSV touchpoint list.

2.4 Authentication and authorisation (lib/auth.js and related)

Function	Behaviour
<code>signAccess(user, { restrict })</code>	HS256 access token with roles, permissions and token version; restricted tokens for two-factor enrolment or a forced password change
<code>signRefresh(user, jti), signToken()</code>	Refresh token (recorded in <code>refresh_tokens</code>); other short-lived tokens (two-factor challenge, quotation approval, report download) with a distinct <code>type</code> claim
<code>verify()</code>	Accepts HS256 only
<code>authenticate(), requireAuth</code>	Token type, user active and unchanged token version; restricted tokens only on their allowed paths
<code>revokeSessions(userId, { refresh })</code>	Raises <code>token_version</code> ; optionally revokes all refresh tokens
<code>requirePermission(...codes), requireRole(...roles), isAdmin(), hasPermission()</code>	Authorisation middleware; <code>it-admin</code> and <code>ba</code> pass every check
<code>lib/password.js</code>	Policy from <code>security.password_*</code> ; history of the last N hashes; expiry after <code>security.password_max_age_days</code>
<code>lib/totp.js, lib/secrets.js</code>	RFC 6238 TOTP (any authenticator app); AES-256-GCM encryption of the TOTP secret; HMAC-hashed reset codes
<code>lib/loginHistory.js</code>	Records every password, two-factor and refresh attempt with IP, user agent and reason

Note: The list of administrator roles (`it-admin`, `ba`) is repeated in several places in the code (code review item 22). Changing which roles are administrators requires a code change.

2.5 Record scope (lib/scope.js)

A user is scoped when every role they hold is listed in `security.scoped_roles` (default `["agent"]`). `scopeOf(req)` returns the scope; list services add `scopeSql(scope, entity, alias, params)` to their WHERE clause; detail and action routes use the `ownRecord(entity)` middleware, which answers 404 for a record the user may not see. Scoped entities: lead, client, quotation, policy, endorsement, claim, renewal, receipt, commission and referrer, matched on the owner, agent or creator columns and on ownership of the client.

2.6 Settings (lib/settings.js)

`getSetting(key, fallback)` reads `app_settings.value` (JSONB) and caches it in a process-level map; `getSettings(group)` lists a group; `setSetting()` updates the row, stamps `updated_by` / `updated_at` and removes the key from the local cache. All business parameters (tax rates, prefixes, limits, templates, e-mail texts, security policy, report options) are read this way; the code review found all setting keys seeded and read through `getSetting`.

Gap: The settings cache has no expiry and is cleared only in the process that saved the change. With two or more API instances, a change made in Master > Configuration reaches the other instances only when they restart; until then they apply the old value (for example a tax rate, a numbering prefix or a rate limit). Recommended: a short time-to-live (for example 30 to 60 seconds) or a PostgreSQL `LISTEN / NOTIFY` invalidation; until then restart all instances after configuration changes.

2.7 Numbering

`next_number(name, prefix)` in SQL returns `<prefix>-<year>-<5 digits>` from the counter table sequences; the prefix is `numbering.<entity>.prefix` (43 keys). Helpers: `nextNumber()` in `modules/documents/common.js` (quotations, policies, endorsements), `modules/claims/util.js` (claims, renewals, campaigns; re-checks uniqueness), `modules/masters/helpers.js` (incentive, reinsurance, remittance items) and `nextNo()` in disbursements. Details and the year / time-zone gap: document 02.

2.8 Audit (lib/audit.js)

`audit(req, { entity, entityId, action, before, after })` inserts into `audit_log` the user id and name, entity, id, action, before / after JSON and the client IP. It is called by mutating handlers after the change (it runs as a separate statement, not inside the business transaction). Reviews: Master > Audit trail (GET `/api/settings/audit`, permission `read:audit`, filters entity, entity id, username; newest 200, at most 1,000).

2.9 Notifications and audience

`notify({ userId, type, priority, title, message, link, entity, entityId, audience })` inserts a row in `notifications`. A notification is addressed to one user or, without a user, to everyone who holds the permission in `audience` (for example `write:journal-vouchers` for journal vouchers awaiting approval); administrators see all audiences. The front end polls the unread count every 30 seconds while signed in (`context/NotificationContext.js`). Notification switches: `notification.approval_requests`, `notification.claim_status`, `notification.renewal_reminder`.

2.10 E-mail outbox (lib/mailer.js)

`queueEmail({ to, cc, subject, html, template, entity, entityId })` inserts into `email_outbox`; nothing is sent in the request. The `email-outbox` job calls `sendQueuedEmails()` every 5 minutes: when `notification.email_enabled` is true and `SMTP_URL` is set, it sends up to 50 queued messages (oldest first) from `notification.from_address`, marks them sent, or increments `attempts` and stores the error; after 5 failed attempts the message is failed. Templates are settings (`email.template.*`, `security.reset_email_*`, `reports.email_*`, `remittance.*_email_*`) with `{{placeholder}}` substitution and HTML escaping (`renderTemplate()`).

Note: With e-mail disabled (the seeded default `notification.email_enabled = false`) or without `SMTP_URL`, messages accumulate in the outbox with status `queued`; they are sent once e-mail is enabled. Throughput is at most

50 messages per 5 minutes (600 per hour) per run; bulk renewal notices use their own queue job (`renewal-queue`, every minute).

2.11 Documents (PDF, XLSX, CSV)

Writer	Use
<code>modules/documents/pdf.js</code>	A4 documents (title, key-value blocks, tables, paragraphs, automatic page breaks) for quotations, policy schedules, receipts, billing statements, claim letters and slips; Helvetica with WinAnsi text (the peso sign is written as PHP)
<code>tools/pdf.js</code>	Landscape tabular report PDFs (repeated header row, zebra rows, totals, page numbers)
<code>tools/xlsx.js + tools/zip.js</code>	Multi-sheet XLSX with styled frozen header, auto-filter, typed cells (money, date, integer)
<code>tools/csv.js</code>	RFC 4180 CSV with UTF-8 BOM and formula-injection guarding

2.12 Reports engine (modules/reports)

The catalogue (`report_definitions`, 19 reports) defines parameters, base query name, default columns, roles and permission. `engine.js` normalises the screen parameters (dates in the business time zone, agent, insurer, branch, client filters), wraps the base query from `queries.js` with parameterised filters, sorting, paging and totals, and validates column identifiers. `service.js` runs reports on screen (`POST /reports/{code}/run`), generates files (CSV, XLSX, PDF; at most `reports.max_rows` = 50,000 rows) into `UPLOAD_DIR/reports` recorded in `generated_reports`, returns download links signed for `reports.download_link_ttl_hours` (72 h), and runs report schedules as scheduled jobs that e-mail a download link. Generated files older than `reports.retention_days` (90) are deleted by the daily-reports job.

2.13 File storage and signed links

Function	Behaviour
<code>storeFile(file, { folder, userId, entity, entityId })</code>	Checks the file content against <code>uploads.allowed_types</code> (signature detection; the detected type is stored), reserves a key <code>folder/<timestamp>-<128-bit random>-<name></code> , writes the file under <code>UPLOAD_DIR</code> and records it in <code>documents</code>
<code>reserveKey(), writeObject()</code>	Two-step upload used by the presigned-URL endpoints (<code>/s3/presigned-upload-url</code> , <code>PUT /s3/put/*</code>)
<code>resolveKey()</code>	Normalises the key and refuses paths outside <code>UPLOAD_DIR</code>
<code>publicUrl(key) / signedUrl(key)</code>	Canonical URL stored in records / signed URL with <code>exp</code> and HMAC <code>sig</code>
<code>lib/fileLinks.js</code>	Middleware that signs every object URL in JSON responses (one expiry per response), so records keep unsigned URLs and links copied out of the application expire
<code>GET /s3/object/*</code>	Serves a file to a bearer-token caller or a valid signature; <code>nosniff</code> , sandbox CSP (PDF excepted), attachment disposition for non-image / non-PDF
Delete / overwrite	Only the uploader, a writer of the owning module or an administrator

The storage functions keep an S3-like key space so the implementation can be replaced by Amazon S3 (`storage.js`: "swap for S3 by replacing these functions"). Some modules write generated files directly under `UPLOAD_DIR` and register them in `documents` (`claims/util.js`, `accounting/lib/files.js`, `masters/helpers.js` `saveFile`) or in `generated_reports` (reports).

2.14 Scheduler (jobs/scheduler.js)

Aspect	Behaviour
Start	<code>startScheduler()</code> at API start (unless <code>SCHEDULER_ENABLED=false</code>) schedules every enabled row of <code>scheduled_jobs</code> with node-cron; invalid cron expressions are skipped with a warning; editing a job or a report schedule reloads the schedule on the instance that handled the change
Single execution	<code>runJob()</code> takes <code>pg_try_advisory_lock(hashtext('brokerverse.scheduled_job'), hashtext(code))</code> on a dedicated connection; if another instance holds it the run is skipped. A scheduled run is also skipped when a scheduled run of the same job already started in the same minute (clock skew between instances)
Recording	Each executed run inserts <code>job_runs</code> (triggered by <code>schedule</code> or the user name) and updates <code>scheduled_jobs.last_run_at / last_status</code> ; the handler output (JSON) or error message is stored
Manual run	Master > Schedules > Run now (POST <code>/api/schedules/{code}/run</code> , administrators)
Handlers	<code>renewalNotices</code> , <code>policyExpiry</code> , <code>quoteExpiry</code> , <code>receivableAgeing</code> , <code>dailyReports</code> , <code>emailOutbox</code> , <code>processRenewalQueue</code> , <code>renewalPipeline</code> , <code>collectionReminders</code> , <code>scheduledReport</code>

Gap: A schedule change reloads the cron tasks only on the instance that served the request; other instances keep the old schedule until restart (the advisory lock still prevents double runs). The scheduler time zone is the container's (UTC), see document 01. There is no alert when a job fails: failures are visible only in Master > Schedules and `job_runs` (document 11).

2.15 Rate limiting and upload limits

`lib/rateLimit.js` keeps two in-memory limiters per process: a sliding window for sign-in and forgot-password (`security.login_rate_limit`, 10 per 300 s, keyed by IP and by username) and a fixed-window global API limit (`security.api_rate_limit`, 600 requests per 60 s per signed-in user or per IP; health checks and CORS pre-flights are not counted; 429 with `Retry-After`). `lib/uploadLimits.js` provides `memoryUpload()` (documents and photos, per-file size and file count) and `importUpload()` (one spreadsheet or statement file), `inflateEntry()` (caps decompression of workbook parts) and `assertRowLimit()`.

2.16 Dates, money and validation

Component	Interface
<code>lib/dates.js</code>	<code>businessTimeZone()</code> (general.timezone), <code>today()</code> , <code>businessDate()</code> , <code>nowInTz()</code> , <code>calendarPeriod(period)</code> , <code>addDays()</code>
<code>lib/money.js</code>	<code>round2()</code> (half away from zero with floating-point correction), <code>formatMoney(amount, currency)</code> using <code>currency.default</code> , its locale and <code>currency.decimals</code>
<code>lib/birthDate.js</code>	Age plausibility of dates of birth (<code>leads.min_age_years</code> , <code>leads.max_age_years</code>)
<code>lib/validate.js</code>	<code>validate(schema, part)</code> middleware with zod; the parsed value replaces the input
<code>lib/errors.js</code>	<code>HttpError</code> and helpers (<code>badRequest</code> , <code>unauthorized</code> , <code>forbidden</code> , <code>notFound</code> , <code>conflict</code>); <code>errorHandler</code> returns <code>{ success: false, message, requestId }</code> , with a generic message for 5xx
<code>lib/respond.js</code>	<code>ok()</code> , <code>created()</code> envelopes; <code>paging()</code> (page / perPage and aliases, perPage capped at 500, default 10); <code>pageMeta()</code>
<code>db/pool.js</code>	<code>query()</code> , <code>one()</code> , <code>many()</code> , <code>withTransaction(fn)</code> (BEGIN / COMMIT / ROLLBACK on one client)

3 Front-end shared components

Component	Responsibility
API access: utility/interceptor.js, utility/sessionRefresh.js, services/*Service.js	Axios instance with the bearer token; a wrapper around <code>window.fetch</code> for services that call fetch directly. On a 401 from a non-auth endpoint, one shared refresh per tab exchanges the refresh token (or picks up a token already rotated by another tab), retries the request, and ends the session with a redirect to <code>/login?session=ended</code> if the refresh fails. <code>REACT_APP_BASE_URL</code> (compiled in) is the API base.
Session: utility/tokenManager.js, logout.js, idleTimeout.js, redux/store.js	Token storage in <code>localStorage</code> ; sign-out clears the session keys and resets the Redux store; idle sign-out after <code>limits.session_idle_minutes</code> with a one-minute warning.
Route guard and menus: routes/ProtectedRoute, utils/menuPermissions.js, utils/canOpen.js	Deny-by-default role-to-menu map; <code>filterMenuForRoles()</code> , <code>isPathAllowed()</code> guard every route; <code>canOpen(path)</code> hides buttons that lead to screens the role cannot open. The server remains the authority (permissions on every API route).
Dialogs and toasts: components/AppDialogs, utility/dialogs.js, components/Toast, utility/toastUtils.js	One shared PrimeReact toast and confirmation dialog mounted in <code>App.js</code> , replacing the browser's alert / confirm (<code>notify*</code> , <code>confirmAction</code>).
System settings and branding: module/SystemSettings/store/systemSettingsSlice.js, utility/applySystemSettings.js	Loads the public system settings (<code>GET /api/system-settings</code> , no sign-in, also used by the sign-in page) and applies theme colours (CSS variables), favicon, logo, language, display currency, date format, phone format and quotation options.
Formatting: utility/dateFormat.js, hooks/useFormatCurrency.js, utility/currencyConverter.js, utility/phoneFormat.js	Dates in <code>general.date_format</code> ; amounts in the configured currency and locale; Philippine mobile number format from settings.
Configuration-driven options: utility/currencyOptions.js, utility/quoteOptions.js, utility/systemCurrencies.js, master-option hooks	Drop-downs filled from settings and master data (currencies, vehicle colours, model-year span, covers) instead of constants.
Notifications: context/NotificationContext.js, components/NotificationDropdown, NotificationBadge	Unread count polled every 30 seconds while signed in; list, mark read, delete.
File upload: components/S3FileUpload	Upload with progress and preview through the <code>/s3</code> endpoints; displays signed URLs returned by the API.
Internationalisation: <code>i18n.js</code> , locales/en.json, th.json	i18next with browser language detection; English complete; Thai partial; Filipino configured without a translation file.
Production console guard: utility/productionConsole.js	Disables <code>console.log</code> / <code>info</code> / <code>debug</code> in production builds.

Note: The code review (`docs/review/CODE_REVIEW.md`) records three HTTP mechanisms in the front end (raw `fetch` in about 30 services, axios interceptor, a separate axios instance for notifications), all covered by the shared refresh. Consolidating them on one API client is a maintainability improvement.

4 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components (this document)	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

5 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (<code>pg_try_advisory_lock</code>) used by the scheduler so a job runs on one API instance at a time.
API	Application programming interface; here the BrokerVerse REST API under <code>/api</code> served by the Node.js backend.
Audit log / audit trail	Table <code>audit_log</code> : who changed which record, when, from which IP, with the before and after values.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (<code>CORS_ORIGINS</code>).
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
CSP	Content Security Policy: HTTP header restricting what a page may load or run.
Endorsement	Change to an issued policy (details, cover, premium or cancellation).
HMAC	Keyed-hash message authentication code; signs file links and hashes one-time codes.
HS256	HMAC-SHA256 JSON Web Token signature algorithm, the only one the API accepts.
JSONB	PostgreSQL binary JSON column type, used for flexible document parts of records.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
Record scope	Rule that users holding only scoped roles (default: agent) see only their own records (<code>lib/scope.js</code>).
Refresh token	Long-lived token (30 days) exchanged for a new access token; rotated on every use.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when <code>SEED_SAMPLE_DATA</code> is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in <code>SMTP_URL</code> .
Token version	<code>users.token_version</code> , carried in every access token; raising it ends all sessions of the user.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.
zod	TypeScript-first schema validation library used to validate request bodies and parameters.